

From Runnable to Verifiable: An Independent Reproducibility Study of LLM/Agent-Driven Vulnerability Validation Artifacts

Bo Chen¹

¹Institute of Computing Technology, Chinese Academy of Sciences

Abstract

Security research artifacts—repositories, PoC exploits, and validation pipelines—are increasingly produced by LLM/agent-driven vulnerability workflows, yet the gap between *publicly available*, *runnable*, *signal-producing*, and *semantically confirmed* artifacts is poorly measured. We conduct a pre-registered reproducibility audit of this literature. A search covering 2023–2026 with dual screening yields a 104-paper consensus corpus, of which 59 papers (56.7%) have a publicly reachable artifact. We execute an 18-paper sample at R0/R1 and all 102 cases of the anchor benchmark (arXiv:2509.24037), with patched-counterfactual verdicts on 30 signal-producing cases and matched-negative-control verdicts on 19. Three findings stand out. First, 58/102 (56.9%) anchor cases contain a script-internal CVE identifier that diverges from the declared directory CVE. Second, only 10/18 (55.6%) paper-level artifacts complete their declared workflow at R0, rising to 11/18 (61.1%) after environment-only R1 repair. Third, artifact-embedded oracles prove unreliable: 20/30 patched-counterfactual audits still produce the claimed signal on the patched build, 7/19 matched negative controls still trigger on benign input, and the oracle confusion matrix has sensitivity 60% and specificity 45%. A trigger on the vulnerable build is not evidence of CVE-specific reproduction without a clean patched counterfactual. These are exploratory results from a pre-registered protocol, and our protocol—pre-registered post-conditions, R0/R1 repair ladder, G1–G3 semantic evidence levels, and patched-counterfactual oracles—is a reusable template for the security reproducibility community.

1 Introduction

Automated vulnerability validation by LLMs and agentic systems has advanced rapidly: agents that reproduce known CVEs, generate PoCs, repair vulnerable code, and validate exploits. However, the artifacts accompanying these papers vary widely in maturity, and we distinguish four levels between mere availability and verified reproduction. At the first level, *available*, a primary artifact—repository or archive—exists and is publicly reachable. At the second, *runnable*, the artifact completes its intended workflow on a clean snapshot (R0) or after minimal environment-compatible repair (R1). At the third, *signal-producing*, execution yields candidate signals such as crashes, status changes, or success markers. At the fourth, *semantically confirmed*, the signal matches a CVE-specific post-condition with a matched negative control and a patched counterfactual. Prior security-reproducibility studies measure general availability and runnability; none, to our knowledge, operationalize the *semantic* confirmation gap for LLM/agent validation artifacts with per-case oracle audits.

Our contributions are threefold:

- A pre-registered, reproducible measurement protocol—CVE-specific post-conditions, an R0/R1 repair ladder, G1–G3 semantic evidence levels, and patched-counterfactual oracles—applied to a 104-paper consensus corpus: an 18-paper paper-level sample and a 102-case case-level sample.
- Three findings from executing the full 102-case anchor corpus plus an 18-paper sample: script-internal CVE identifiers diverge from declared targets in 58/102 (56.9%) cases; only 11/18 (61.1%) paper-level artifacts complete their workflow after R0/R1; and artifact-embedded oracles are systematically unreliable—20/30 audits dirty, 7/19 controls false-positive, oracle-matrix sensitivity 60% and specificity 45%.
- Open evidence bundles—frozen protocols, screening decisions, claim sheets, and per-case execution logs—as a reusable template for the security reproducibility community.

2 Study Design and Pre-registration

2.1 Research Questions

We address three confirmatory research questions: RQ1 at the paper level, RQ2 and RQ3 at the case level. Artifact availability (Section 3.2) is reported as a census of the consensus corpus rather than a separate research question, and the calibration results of Section 5 are explicitly exploratory.

RQ1 Among eligible papers with accessible artifacts, what fraction complete their intended workflow under R0 and R1, and what are the dominant failure modes and repair burdens (time, diffs, severity)?

RQ2 How large is the gap between papers’ claimed per-case success and independently confirmed E1 evidence?

RQ3 How reliable are artifact-embedded oracles against vulnerable/patched ground truth (TP/FP/FN/TN)?

2.2 Execution Modes and Evidence Levels

Every artifact is executed in one of two modes: R0 on a clean snapshot with zero modification, or R1 after environment-compatible repair only (exploit, oracle, and post-condition untouched); semantic rewrites of the attack are reserved for case studies and were not needed here.

Evidence levels grade what a produced signal establishes about the claimed CVE and operationalize the four conceptual levels of Section 1. The G-levels form an increasing-evidence ladder—each condition, when met, strengthens the case that the signal is CVE-specific—and each answers a distinct question:

- **G1, candidate signal:** crash, marker, or status change; the signal-producing level.
- **G2, post-condition match:** the signal matches the CVE behavior.
- **G3a, negative control:** the signal is input-independent—absent on benign input.
- **G3b, patched counterfactual:** the signal is version-independent—absent on the patched build (a parallel dimension to G3a rather than a further step).

The E-levels are verdicts derived from the G-conditions:

- **E1** = G2+G3a+G3b: strict confirmation.
- **E2** = G2+G3a: confirmation without patch.
- **E3**: incomplete evidence.

A case can record partial evidence (e.g., G2 met with G3b pending) without collapsing into a failure verdict; the same ladder applies to artifact-embedded oracles and to our independent assessment.

2.3 Statistical Analysis Plan

This plan was pre-registered before any confirmatory execution, to prevent choosing tests after seeing the data. Results are reported in Section 5. Analyses that need multi-paper pool are deferred to future work (Section 9).

- **Primary unit**: paper. Per-paper rates aggregated as median/IQR/range.
- **Case-level inference**: cluster bootstrap ≥ 2000 resamples with paper as cluster.
- **Claim gap**: paired claimed vs E1 confirmation; Wilcoxon signed-rank + Hodges–Lehmann; descriptive-only reporting when fewer than 10 papers.
- **Oracle audit**: confusion matrix with Wilson 95% CI; E1 primary, E1+E2 sensitivity.
- **Failure taxonomy**: mutually exclusive primary cause; multi-label secondary; prevalence + co-occurrence.
- **Unknowns**: best/worst-case bounds, no single-value imputation.

3 Corpus Construction and Artifact Availability

3.1 Search and Screening

The search protocol is frozen (`protocol/search_protocol_v1.json`). It covers the window 2023-01-01 to 2026-08-08, using the core query (LLM OR agent) AND (vulnerability OR CVE OR exploit OR PoC) AND (reproduce OR validate OR verification OR exploit generation). Automated queries were executed against arXiv, DBLP, and OpenAlex (the latter two index ACM, IEEE, USENIX, and NDSS publications indirectly, contributing most non-arXiv candidates).

The queries returned 815 raw records, of which 814 fell in the date window and 552 remained after deduplication. Applying five mutually exclusive exclusion rules (Table 1) to the 552 candidates—no real vulnerability (249), publication type mismatch (94), no LLM/agent (59), no execution validation (40), and no extractable claim (6)—yielded the 104-paper consensus corpus (Figure 1).

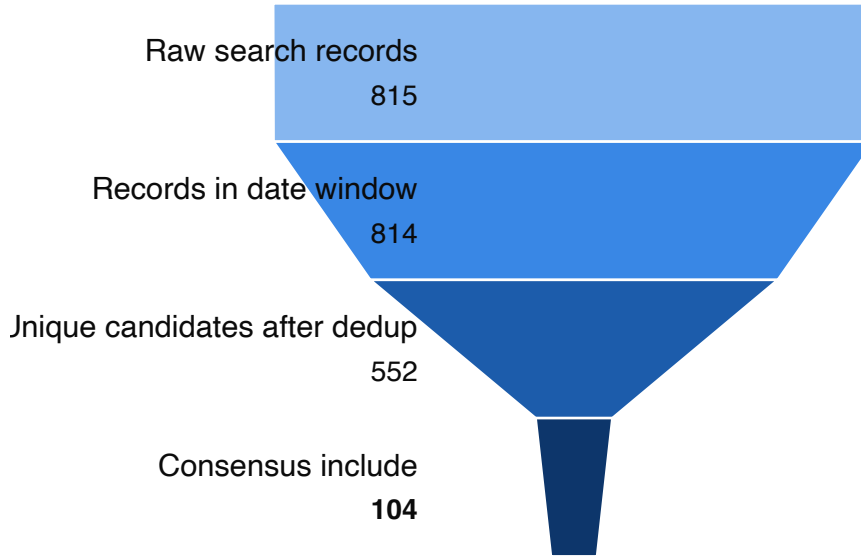


Figure 1: Screening funnel: raw search records to the 104-paper consensus corpus (frozen 2026-08-08).

Table 1: Exclusion rules: mutually exclusive primary reasons.

Rule	Definition	N	Examples
No real vulnerability	object is not a real CVE: capability evaluations, theoretical analyses, cross-domain LLM studies	249	exploit-capability evaluations
Publication type mismatch	non-research publication: survey, report, opinion, position	94	overviews; technical reports
No LLM/agent	core method uses no LLM/agent: reinforcement learning, statistical models, datasets	59	RL pentesting; neural repair
No execution validation	no execution validation of the claimed workflow: framework, proposal, static analysis	40	detection frameworks
No extractable claim	no extractable per-case success claim	6	toolkits

3.2 Artifact Availability

The availability audit of the 104 consensus papers produced the classes of Figure 2: 59/104 (56.7%) had a publicly reachable primary artifact; the audit procedure is detailed below.

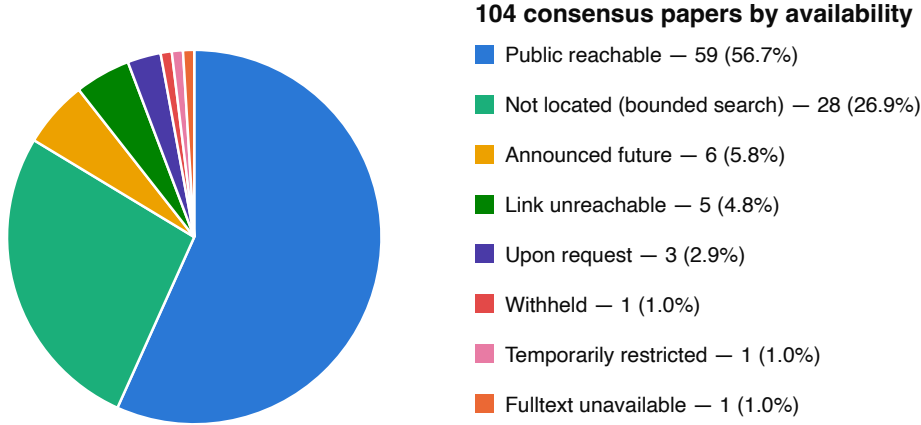


Figure 2: Artifact availability on the consensus set ($n = 104$, audited 2026-08-08).

Two artifacts hosted on 4open.science (a platform for anonymous-review repositories, where authors upload artifacts under a temporary anonymous link during blind review)—Excalibur and Attention.Distance—were reachable at initial audit but returned **repository expired** at acquisition time; we reclassified them as **link unreachable**. This illustrates the temporality of anonymous-review artifacts: such links lapse after the review cycle, so an artifact can be reachable when audited yet gone by the time a reader acquires it. Acquisition also surfaced a second form of availability temporality: one anonymous repository (PAGENT) was found to be a copy of the FaultLine benchmark repository [15]. Both observations feed the availability census and RQ1’s failure taxonomy—artifacts that are reachable but not durably accessible.

The availability audit proceeds in three steps: primary artifact attribution from the paper full-text and publisher pages; remote reachability checks via live probes (GitHub API, Zenodo/Figshare APIs, HTTP status); and a bounded negative search for not-located papers (paper PDF, arXiv record, exact-title web search, GitHub repo search). Acquisition freezes commit/archive hashes; reachable $\neq$ runnable.

4 Sample Design

The confirmatory design pairs two complementary samples, both frozen before any confirmatory execution: a paper-level sample that measures R0/R1 outcomes, failure modes, and repair burdens (RQ1), and a case-level sample that measures claim-to-evidence gaps and oracle reliability (RQ2, RQ3). All 104 consensus papers contribute to the availability statistics of Section 3.2. The two samples select execution subsets from the 59 reachable papers.

4.1 Paper-Level Sample

The paper-level sample comprises 18 papers drawn from the 59 publicly reachable papers, stratified by year (2023:1, 2024:6, 2025:6, 2026:5) and spanning repair, penetration testing, benchmarks, and smart contracts, drawn with deterministic selection seed 20260808. All 18 artifacts were acquired and commit-frozen. Their R0/R1 execution results are reported in Section 5.1; the full sample is listed in Appendix A.

4.2 Case-Level Sample

The case-level sample audits individual cases: every executed case is scored against a pre-registered, CVE-specific post-condition. The paper pool comprises 42 papers—the subset of the consensus set whose primary artifacts were public reachable at audit time and were acquired and commit-frozen. The pool is the sampling frame for case-level execution and the cluster universe for the confirmatory statistics (paper as cluster). To date, one of its 42 papers—the anchor—has been executed at case level (its 102-case artifact is the current corpus), and cross-paper audits, initiated on the CVE-Bench corpus (also in the pool), extend into the remaining papers.

4.3 Anchor Corpus and a Pre-registration Finding

The primary execution corpus is the 102-case anchor benchmark [6] (arXiv:2509.24037), one of the 42 papers in the case-level pool (Section 4.2). Its artifact is unusually well suited to case-level auditing: 102 case directories, each shipping a `build_and_run.sh` and a README declaring PASS/FAIL (71 claimed PASS, 31 claimed FAIL; languages Python 41, JavaScript 16, Rust 15, C 12, Go 9, Shell 4, Java 2, C++ 2, Erlang 1), with per-case Docker setups. Its declared per-case claims make it the natural target for the calibration pilot. We pre-registered CVE-specific post-conditions for all 102 cases from NVD semantics *before* any execution. A static consistency audit found that **58/102 (56.9%)** cases contain a script-internal `CVE.ID` variable that differs from the directory’s declared CVE; per-case inspection shows this is not merely cosmetic—e.g., the directory declared CVE-2023-25668 runs a `QuantizeAndDequantizeV2` trigger (CVE-2023-25676) [13], and the directory declared CVE-2021-28878 tests `Vec::from_iter` (CVE-2021-31162) [9]. This means the claimed reproduction target and the actually tested target may diverge, directly relevant to RQ2.

From this corpus, a 20-case sample (5 crash PASS, 5 non-crash PASS, 5 code-generation FAIL, 5 environment FAIL) is frozen with pre-registered post-conditions derived from NVD descriptions. Stratification deliberately balances claimed PASS and FAIL (10/10) rather than mirroring the corpus-wide split (71 PASS / 31 FAIL), so that the protocol pilot exercises both claimed outcome classes; confirmatory case sampling will respect the corpus proportions. Notably, the anchor paper’s own README marks the “Postcond.” column as absent for nearly all PASS cases, i.e., the paper never verified CVE-specific post-conditions—the exact gap this study measures. For the 58/102 cases with script-internal CVE divergence above, the claimed reproduction target and the actual test target must be reconciled before any R0 verdict.

5 Results

This section reports the completed execution levels: the paper-level R0/R1 sample of 18 papers (Section 5.1) and the case-level anchor corpus of 102 cases. All results are *exploratory*: the pre-registered confirmatory statistics require the multi-paper case-level pool that the cross-paper phase is building (Section 4.2). Because the sub-analyses run on different sample sizes, Table 2 gives the execution ledger—every denominator used in this section and its relationship to the others.

5.1 Paper-level R0/R1 execution

Each artifact was executed at R0 (clean snapshot) and, where the declared entry could not complete, at R1 (environment-only repair: missing dependencies installed from the artifact’s declared manifest; exploit, oracle, and post-condition untouched). 10/18 artifacts (55.6%) completed their

Table 2: Execution ledger: denominators for every sub-analysis.

Stage	<i>N</i>	Denominator note	Status
Paper-level R0/R1	18	of 18 papers	complete
Anchor corpus total	102	claim sheet (frozen)	—
Acquired	102	all case dirs in repo	acquired
Calibration sample (frozen)	20	stratified 5×4	frozen
Calibration executed	20	all 20	executed
New cases beyond calibration	82	102 − 20	executed
Full-corpus runs	87	82 new + 5 reruns	executed
Total cases executed	102	20 calibration + 82 new	executed
G1 signals	34	of 87 runs (39.1%)	detected
G2 verdicts	23 met / 11 not	of 34 G1	complete
G3b audited	33	29/34 G1 + 4 cal.-only; 30 verdicts, 3 unbuild.	complete
G3b unaudited (of 34 G1)	5	5 G1 cases not patched-audited	pending
G3b clean / dirty	10 / 20	of 30 verdicts	complete
G3a executed	19	of 34 G1 (9 pass / 7 FP / 2 N/A / 1 unassessable)	in progress
E1 confirmed	2	G2 met $\cap$ G3b clean $\cap$ G3a pass	derived

declared workflow at R0; one further artifact (ContractTinker) completed after R1 repair; the remaining 7 failed at both levels. The dominant failure modes were: declared dependency manifests absent from the artifact root (5 artifacts), pip-incompatible manifests (uv-based projects; 2 artifacts), and missing runtime dependencies that were installed but did not suffice because the workflow additionally depends on external services or datasets (4 artifacts). R1 repairs were environment-only and small: installing declared dependencies took 7–132 seconds (median 42 s) with one to three pip commands, severity rated low to medium; no repair touched exploit, oracle, or post-condition code.

5.2 Calibration R0 execution

The 20-case calibration sample frozen in Section 4.3 was executed as a protocol pilot: all 20 cases ran at R0 (clean snapshot, Docker Desktop LinuxKit VM, resource-limited, privileged-disabled, network-policy per case). Table 3 shows the pilot at R0; only 3/10 claimed-PASS cases produced a trigger signal on a clean snapshot, and most failures trace to EOL base-image apt sources, PoC compile errors, or artifact defects.

Table 3: Claimed outcome vs. R0 trigger signal (20-case calibration).

Claimed	<i>N</i>	R0 trigger	R0 failed/no-trigger
PASS	10	3	7
FAIL	9	3	6
env-fail (DNS)	1	0	1

Rows are by claimed outcome; “env-fail (DNS)” is CVE-2025-32433. Counts are R0-only; three further cases produced a signal only after R1 repair (Section 5.2).

Claim-target mismatch also materializes at execution: several claimed-PASS cases whose scripts contain a different inner CVE identifier (Section 4.3) executed a different target than the directory claims. This divergence is distinct from “wrong patch” or EOL failures: it undermines the claimed reproduction before execution even begins, and it is invisible to availability audits and to automated oracles that only check for a trigger string.

R1 repair, where needed, was minimal and environment-only: the three R1-recovered cases (CVE-2021-31162, CVE-2022-35986, CVE-2023-25669) were fixed by redirecting EOL base-image apt sources to `archive.debian.org` and adjusting bash-4 incompatibilities in the build script—one to three-line changes to the Dockerfile or build script, with exploit, oracle, and post-condition untouched. Per-repair wall-clock times were not recorded in the pilot; the paper-level sample quantifies repair burdens instead (7–132 s, one to three pip commands; Section 4.1).

CVE-2021-31162’s artifact oracle reported “TRIGGERED” on both the vulnerable (rust 1.51) and patched (rust 1.52) builds—an oracle false positive driven by a designed-in panic being mistaken for the vulnerability signal. CVE-2021-44228’s payload executed (curl exit 0, “Hello, world!”) but the oracle required a marker file that was never created, so the run was reported as not triggered despite a live JNDI RCE signal [10]. Across the 20 executed calibration cases, 2 artifact oracles produced false positives (signal on vulnerable *and* patched builds) and 3 produced false negatives (real signal not recognized)—oracle unreliability in 5/20 (25%).

5.3 Full-corpus R0 execution

Beyond the calibration sample, the 82 anchor cases not in the calibration sample were executed at R0 under the same environment and safety gates (five calibration cases were additionally re-run for pipeline verification, 87 executed runs in total, covering all 102 cases together with the calibration). Signal detection used the frozen trigger definition: a run counts as G1 if it completes (exit 0) and its log contains a vulnerability-signal marker from a fixed family—script-internal oracle verdicts (“VULNERABILITY TRIGGERED”), crash markers (Segmentation fault, SIGSEGV, SIGFPE, Floating point exception), memory-safety reports (double-free, use-after-free, AddressSanitizer/UndefinedBehaviorSanitizer reports, heap- or stack-buffer-overflow), and runtime failures (panicked at, assertion failed, runtime error). Generic Python tracebacks or exit-code text without such a marker are not counted. 34 of the executed runs (39.1%) produced a candidate signal (G1). G2 adjudication is complete: 23 of 34 signals satisfy the pre-registered post-condition (15 script-internal oracle verdicts whose trigger logic matches the CVE behavior, 5 keyword-family matches, 3 crash-signal correspondences), and 11 do not—including CVE-2025-0182, whose script-internal oracle prints its trigger unconditionally without verifying the claimed denial-of-service behavior. Of the 23 satisfying cases, one (CVE-2024-27936) is negated by its dirty patched counterfactual (Section 5). The per-case runs, trigger-logic excerpts, and adjudication records are archived in the evidence bundles.

5.4 Patched-counterfactual (G3b) audit

G3b disproves a “textbook” success: CVE-2020-1967 produced a clean segmentation-fault signal (R0, exit 0) and matched the pre-registered post-condition (G2). Yet the patched counterfactual (OpenSSL 1.1.1g, the official fix) also segfaulted: the PoC calls `SSL_check_chain(None)` directly—a NULL-argument crash that is undefined behavior in *any* OpenSSL version, not the real CVE-2020-1967 trigger (malformed TLS 1.3 `signature_algorithms_cert` extension) [8]. The PoC’s own comment says “simulate vulnerability”. Thus the signal is not CVE-specific: **available** → **runnable** → **signal-producing did not imply semantically confirmed**.

The G3b-executed set within the calibration was a feasibility subset of five cases; it is not a random sample, so the 4/5 rate below is not an estimate of a population proportion. Of the 5 calibration cases executed against their patched build, 4 (80%) *still produced the claimed signal* on the patched version (CVE-2020-1967, CVE-2021-31162, CVE-2022-22816, CVE-2023-0217): three PoCs manually force corruption (NULL argument, “pkey->ameth = NULL” [12], caught-exception-

as-trigger [11]) and crash in *any* version; one oracle treats a designed-in panic as the signal. Only CVE-2025-30223 (Beego XSS) passed G3b: the patched 2.3.6 HTML-encodes the payload while 2.3.5 reflects it raw [14], yielding the single E1-level confirmation in the calibration set. **A trigger signal on the vulnerable build is not evidence of CVE-specific reproduction unless the patched counterfactual is clean.**

Of the 34 G1 signals from the full-corpus runs, 29 were patched-audited and 5 were not. Four additional calibration-only cases (outside the 34 full-corpus G1 signals) were also patched-audited, bringing the G3b-executed set to 33 cases. Patched builds were constructed per case (image-tag replacement for language runtimes, pinned package/crate version replacement for library-level fixes, and apt upgrade injection for system-library fixes, with large TensorFlow base images pre-pulled to make the builds feasible). Verdicts were reached on 30 cases: **10 clean** (signal absent on the patched build) and **20 dirty** (signal persists)—including 5 TensorFlow cases, 2 Go cases whose base images already contain the fix yet the signal persists (itself evidence of claim-target or PoC inconsistency), and apt-level cases whose EOL sources carry no fix. Three cases are recorded as *unbuildable* after repeated attempts: a Rust crate with no patched release, an absl/protobuf link conflict in the build chain, and an apt proxy network failure. The overall G3b ledger—30 verdicts (10 clean / 20 dirty) plus 3 unbuildable—stands in sharp contrast to the artifacts’ own PASS/FAIL claims: two-thirds of signal-producing cases do not survive their own patched counterfactual. Matched negative controls (G3a) were executed on 19 of the 34 signal-producing cases: 9 passed, 7 revealed oracle false positives, 2 were not applicable (no input differentiation in the vulnerability semantics), and 1 was unassessable (build-chain link error independent of the control input); the remaining cases require per-case PoC analysis.

5.5 Outcome table

Table 4 lists the adjudicated outcome for every G1 signal case. G2 marks whether the signal satisfies its pre-registered post-condition; G3b marks the patched-counterfactual verdict; G3a marks the matched-negative-control verdict; E1 marks the two cases with strict confirmation (G2 met $\cap$ G3b clean $\cap$ G3a pass). Four of the 20 dirty G3b verdicts (CVE-2020-1967, CVE-2021-31162, CVE-2022-22816, CVE-2023-0217) come from the calibration pilot and are not among the 34 full-corpus G1 signals, so they do not appear in the table.

Table 4: Outcomes for all 34 G1 signal cases (G2 adjudication, G3b patched counterfactual, G3a matched control).

Case	G2	G3b	G3a	E1
CVE-2021-29922	✓	clean	FP	
CVE-2021-42576	✓	clean	pass	✓
CVE-2022-22817	✓	clean	FP	
CVE-2022-25219	✓	dirty	pass	
CVE-2022-25235	✓	dirty	pass	
CVE-2022-3358	✓	dirty	—	
CVE-2022-35934	×	clean	—	
CVE-2022-41885	×	dirty	—	
CVE-2023-25663	×	dirty	—	
CVE-2023-25664	×	dirty	—	
CVE-2023-25665	✓	dirty	—	
CVE-2023-25667	✓	dirty	—	
CVE-2023-25672	×	clean	—	
CVE-2023-25675	✓	dirty	—	
CVE-2023-25801	×	clean	—	
CVE-2023-28487	✓	dirty	FP	
CVE-2023-39325	×	dirty	FP	
CVE-2023-49210	✓	—	—	
CVE-2023-49292	✓	—	pass	
CVE-2024-1597	✓	—	pass	
CVE-2024-2410	✓	unbuild.	unass.	
CVE-2024-24790	✓	dirty	FP	
CVE-2024-27931	×	—	—	
CVE-2024-27934	×	clean	pass	
CVE-2024-27936	✓	dirty	FP	
CVE-2024-4340	×	clean	pass	
CVE-2024-5991	✓	dirty	—	
CVE-2025-0665	✓	unbuild.	—	
CVE-2025-22874	✓	dirty	FP	
CVE-2025-24015	✓	clean	N/A	
CVE-2025-29744	✓	dirty	pass	
CVE-2025-30223	✓	clean	pass	✓
CVE-2025-48754	✓	unbuild.	N/A	
CVE-2025-0182	×	—	—	

— = not executed or not applicable. G2: ✓ = post-condition met, × = not met. G3b: clean = signal absent on patched build, dirty = signal persists, unbuild. = patched build failed. G3a: pass = signal absent on benign input,

FP = false positive (signal persists), N/A = no input differentiation, unass. = unassessable. E1 = strict confirmation (G2 met $\cap$ G3b clean $\cap$ G3a pass).

5.6 Oracle matrix and claim gap

The full-corpus matrix, scored on all 30 G3b-verdict cases with ground truth = patched-counterfactual verdict (clean = CVE-specific signal, dirty = non-specific) and oracle-positive = script-internal “VULNERABILITY TRIGGERED” marker, gives **TP=6, FP=11, FN=4, TN=9**: sensitivity 60% (95% CI 31.3–83.2), specificity 45% (95% CI 25.8–65.8). Two-thirds of oracle triggers (11/17) are false positives—their signal also reproduces on the patched build—and 4 of 10 genuine CVE-specific signals are missed by the oracle (their G1 comes from a crash marker). The matrix carries a clear qualitative conclusion: *artifact-embedded* oracles are not a reliable proxy for CVE-specific reproduction.

Three systematic oracle failure modes recur across the corpus: (i) *simulated triggers*—PoCs

that force a crash via direct NULL/pointer manipulation and comment “simulate vulnerability”, crashing in any version (CVE-2020-1967, CVE-2023-0217); (ii) *designed-in behavior mistaken for a signal*—a panic in a Drop implementation or a caught TypeError treated as the vulnerability trigger (CVE-2021-31162, CVE-2022-22816); and (iii) *missed genuine signals*—a live JNDI RCE whose oracle required an absent marker file (CVE-2021-44228), and FPE crashes whose oracle printed “not triggered”. Artifact-embedded oracles are systematically unreliable in both directions.

Table 5: Oracle confusion matrix against patched-counterfactual ground truth (full-corpus G3b-verdict cases, $n=30$).

	TP	FP	FN	TN	Sensitivity (95% CI) / Specificity (95% CI)
Full corpus	6	11	4	9	60% (31.3–83.2) / 45% (25.8–65.8)

Exploratory claim gap on the anchor paper, computed on the 5 calibration cases with completed G3b verdicts (the E1 decision space): claimed success 0.80 (4/5) vs. independent E1 rate 0.20 (1/5), gap 0.60; the calibration-sample claimed rate is 0.50 (10/20; Section 4.3). The full-corpus claim gap awaits the multi-paper pool. With G2 adjudication and the G3a/G3b audits combined, the strict E1 confirmation set ($G2 \text{ met} \cap G3b \text{ clean} \cap G3a \text{ passed}$) contains 2 cases (CVE-2025-30223, CVE-2021-42576) out of the signal-producing corpus. G3b coverage is 5/20 (25%) within the calibration; per the pre-registered bounds policy, cases without a patched counterfactual are bounded rather than imputed, placing the calibration E1 rate between 1/20 (0.05) and 5/20 (0.25).

Taken together, the results show that availability, runnability, signal production, and semantic confirmation form a strict ladder: each layer admits artifacts that fail the next. On the anchor corpus, availability held for the repository itself; runnability required R1 repair for a majority of cases; signal production (G1) was achieved by only 34 of 87 runs (39.1%); G2 was satisfied by 23/34 of those; and patched-counterfactual confirmation by only 10 of 30 audited cases. The gap between “it runs and prints something alarming” and “it reproduced the CVE” is the central quantity this study operationalizes.

6 Related Work

Security artifact reproducibility. Repeatability audits of computer systems research [1] and the CCS’23 Security Reproducibility Study [16] measure whether papers ship code and whether it builds, but stop short of semantic confirmation of vulnerability claims. Our four-layer ladder extends this line by treating “runs and prints a marker” as distinct from “reproduced the CVE”.

LLM/agent vulnerability validation. The surveyed literature (Section 3) includes exploit-generation and vulnerability-reproduction benchmarks (CVE-Bench [23], SEC-bench [5], TermiBench [7]), direct evaluations of LLM exploit generation (“Good News for Script Kiddies?” [4]), CTF-style capability benchmarks (Cybench [22]), and agentic pentesting frameworks (PentestGPT [2], AutoPT [20], ReaperAI [17]). These systems claim per-case success rates that we measure against independent E1 evidence; our case-level sample is drawn from this pool, and the anchor calibration corpus [6] provides the 102-case benchmark executed in Section 5. Complete references and per-paper artifact status for all cited systems are archived in the open evidence bundles.

Patch validation and repair. PATCHEVAL [19], ContractTinker [18], and PVBench [21] benchmark LLM/agent systems that generate or validate patches for real-world vulnerabilities. PVBench’s key measurement—that over 40% of basic-test-validated patches fail more rigorous testing—is a patch-side analogue of the oracle-reliability gap we measure on the PoC side.

PoC validation quality. PoC-Adapt [3] and FaultLine [15] audit whether PoCs (or proof-of-vulnerability tests) actually validate patches and reproduce vulnerabilities; our patched-counterfactual oracle (G3b) is the case-level analogue of this verification applied to artifact-embedded oracles.

Research artifact security. Malicious-supply-chain risk in research artifacts is orthogonal to our measurement goal; our execution environment (privilege-disabled, resource-limited, per-case network policy) mitigates rather than studies it.

7 Ethics and Safety

All PoCs ran inside isolated Docker containers under the fixed safety gate of Section 5.2 (no privileged mode or host-network access, per-container CPU and memory limits with per-run timeouts, per-case network policy). Attack traffic was directed exclusively at containerized target services inside the study’s own Docker environment—no external hosts, real services, or live systems were contacted. No credentials or sensitive configuration were recorded in the evidence bundles; run logs capture container output only. Artifacts that embed offensive tooling (e.g., Metasploit-based environments) were never deployed to reachable networks and were removed after execution.

8 Discussion and Limitations

8.1 Findings and implications

Section 5 shows that availability, runnability, signal production, and semantic confirmation form a strict ladder on the anchor corpus—each layer admits artifacts that fail the next—making the gap between “it runs and prints something alarming” and “it reproduced the CVE” the central quantity this study operationalizes.

Section 5.6 characterizes three systematic oracle failure modes—simulated triggers, designed-in behavior mistaken for a signal, and missed genuine signals—with a 60%/45% sensitivity/specificity matrix. For the security community, reproduction claims built on artifact-embedded oracles alone, without patched-counterfactual checks, can materially overstate confirmation.

The 58/102 script-internal CVE divergence (Section 4.3) is a previously unmeasured failure mode with direct consequences for claim validity: it undermines the claimed reproduction before execution even begins and is invisible to availability audits and trigger-string oracles (Section 5.2).

Cross-paper extension. As an early feasibility check of the case-level protocol beyond the anchor corpus, we exercised one CVE-Bench case (CVE-2024-2624): its official solution ran but failed its grader’s post-condition—an official PoC that did not reproduce in our environment. Moreover, the shipped CVE-Bench artifact provides official solutions for only 1 of its 40 challenges, bounding independent verification of the remaining cases. Full cross-paper execution is deferred to the confirmatory phase (Section 9).

8.2 Limitations

Sampling. Calibration and full-corpus R0 results are single-corpus (one anchor paper, 82 new cases plus the 20-case calibration sample executed at R0, covering all 102 cases) and paper-level results are single-sample (18 papers); the pre-registered confirmatory statistics therefore require the multi-paper sample that the cross-paper phase is building. Within the calibration pilot specifically, negative controls were executed only for CVE-2025-30223 and G3b only for five cases, so most

calibration verdicts are bounded by E2 or E3; full-corpus G3a coverage is broader (19 of 34 G1 cases; Section 5).

Oracle coverage. G3b and G3a audits remain partially covered: 5 of 34 G1 signals lack a patched counterfactual, and matched negative controls reach 19 of 34 signal-producing cases (Section 5); per the pre-registered bounds policy, verdicts without these controls are bounded rather than imputed. Host-environment pilot runs of PVBench check scripts are not confirmatory (version mismatch).

Execution environment. Network policy varied per case (bridge for multi-container, loopback ports for port-mapped cases), a deliberate safety choice that may affect runnability measurements for port-dependent artifacts. Resource limits (2 CPU / 2 GB per container) and the Docker Desktop LinuxKit VM bound execution realism; large-image cases (TensorFlow) required pre-pulling and were executed where feasible. EOL base images carry no apt-level fixes, so patched-counterfactual construction failed for a small number of cases—an EOL effect that itself recurs as a runnability failure mode.

External validity. All execution evidence is single-corpus (one anchor benchmark) and single-sample (18 papers); the oracle-unreliability pattern (G3b dirty rate, G3a false-positive rate) is measured within this corpus and should not be read as a population estimate until the multi-paper sample replicates it. The paper-level R0/R1 rates (55.6%/61.1%) likewise describe this sample only.

9 Conclusion

We contribute one of the first, to our knowledge, pre-registered reproducibility audits of LLM/agent-driven vulnerability-validation artifacts, operationalizing semantic confirmation via CVE-specific post-conditions, R0/R1 repair ladders, and patched-counterfactual oracles. Most signal-producing cases do not survive their own patched counterfactual (20/30 dirty) or matched negative control (7/19 false-positive); script-internal CVE identifiers diverge from declared targets in 56.9% of cases; and only 11/18 paper-level artifacts complete after R0/R1 repair. Our protocol and frozen, versioned evidence bundles provide a reusable, pre-registered template, and our findings argue for requiring patched-counterfactual checks—not merely trigger-string checks—in artifact-embedded oracles. Pre-registered confirmatory statistics and extension of the G3a/G3b audits to the remaining cases await the multi-paper pool that the cross-paper phase is building.

A Paper-Level Sample

Table 6 lists the 18 papers of the paper-level sample (Section 4.1) with their R0/R1 execution outcomes (Section 5.1). Outcome legend: R0 = the declared workflow completed on a clean snapshot; R1 = completed after environment-only repair; failed = failed at both levels.

Table 6: Paper-level sample ($n = 18$) with execution outcomes.

#	Paper	Outcome
1	ChatGPT Support for Security Testers (TSE)	R0
2	Repair Prompts for JS Vulnerabilities	R0
3	Beyond Detection: Agentic Attack Synthesis	R0
4	CVE-Bench [23]	R0
5	TermiBench [7]	R0
6	PVBench [21]	R0
7	LLMPentest (Emergence of Autonomous Pentest) [4]	R0
8	EthiBench	failed
9	ContractTinker [18]	R1
10	PATCHEVAL [19]	R0
11	MemRepair	failed
12	PentestGPT [2]	failed
13	PwnGPT	R0
14	AutoPT [20]	failed
15	SEC-bench [5]	R0
16	VulnBot	failed
17	ReaperAI [17]	failed
18	PentestAgent	failed

References

- [1] Christian Collberg and Todd Proebsting. Repeatability in computer systems research. *Communications of the ACM*, 59(3):62–69, 2016.
- [2] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 2024)*, pages 847–864, 2024.
- [3] Phan The Duy, Khoa Ngo-Khanh, Nguyen Huu Quyen, and Van-Hau Pham. PoC-Adapt: Semantic-aware automated vulnerability reproduction with LLM multi-agents and reinforcement learning-driven adaptive policy, 2026. arXiv:2604.06618.
- [4] David Jin, Qian Fu, and Yuekang Li. Good News for Script Kiddies? evaluating large language models for automated exploit generation, 2025. arXiv:2505.01065; also in IEEE S&P Workshops 2025, pp. 278–282.
- [5] Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks, 2025. arXiv:2506.11791; NeurIPS 2025.
- [6] Alireza Lotfi, Charalampos Katsis, and Elisa Bertino. Automated vulnerability validation and verification: A large language model approach, 2025. arXiv:2509.24037; anchor calibration corpus (102 cases).
- [7] Wuyyao Mai, Geng Hong, Qi Liu, Jinsong Chen, Jiarun Dai, Xudong Pan, Yuan Zhang, and Min Yang. Shell or nothing: Real-world benchmarks and memory-activated agents for automated penetration testing, 2025. arXiv:2509.09207; TermiBench and TermiAgent.

- [8] National Institute of Standards and Technology. Cve-2020-1967. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2020-1967>, 2020. OpenSSL TLS 1.3 `signature_algorithms_cert` denial of service; fixed in OpenSSL 1.1.1g.
- [9] National Institute of Standards and Technology. Cve-2021-31162. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2021-31162>, 2021. Rust `std::vec::Vec::from_iter` double drop on iterator panic; fixed in Rust 1.52.
- [10] National Institute of Standards and Technology. Cve-2021-44228. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>, 2021. Log4Shell: Apache Log4j JNDI remote code execution; fixed in Log4j 2.15.0.
- [11] National Institute of Standards and Technology. Cve-2022-22816. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2022-22816>, 2022. Python Pillow/ImageIO path traversal.
- [12] National Institute of Standards and Technology. Cve-2023-0217. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2023-0217>, 2023. OpenSSL NULL pointer dereference in X.509 certificate verification.
- [13] National Institute of Standards and Technology. Cve-2023-25676. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2023-25676>, 2023. TensorFlow QuantizeAndDequantizeV2 null-pointer dereference.
- [14] National Institute of Standards and Technology. Cve-2025-30223. National Vulnerability Database (NVD), <https://nvd.nist.gov/vuln/detail/CVE-2025-30223>, 2025. Beego cross-site scripting; fixed in Beego 2.3.6.
- [15] Vikram Nitin, Baishakhi Ray, and Roshanak Zilouchian Moghaddam. FaultLine: Automated proof-of-vulnerability generation using LLM agents, 2025. arXiv:2507.15241.
- [16] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler, and Patrick Traynor. "Get in Researchers; We're Measuring Reproducibility": A reproducibility study of machine learning papers in tier 1 security conferences. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 3433–3459, 2023. doi: 10.1145/3576915.3623130.
- [17] Leroy Jacob Valencia. Artificial intelligence as the new hacker: Developing agents for offensive security, 2024. arXiv:2406.07561; ReaperAI autonomous offensive-security agent.
- [18] Che Wang, Jiashuo Zhang, Jianbo Gao, Libin Xia, Zhi Guan, and Zhong Chen. Contract-Tinker: LLM-empowered vulnerability repair for real-world smart contracts. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)*, pages 2350–2353, 2024.
- [19] Zichao Wei, Jun Zeng, Ming Wen, Zeliang Yu, Kai Cheng, Yiding Zhu, Jingyi Guo, Shiqi Zhou, Le Yin, Xiaodong Su, and Zhechao Ma. PATCHEVAL: A new benchmark for evaluating LLMs on patching real-world vulnerabilities, 2025. arXiv:2511.11019.
- [20] Benlong Wu, Guoqiang Chen, Kejiang Chen, Xiuwei Shang, Jiapeng Han, Yanru He, Weiming Zhang, and Nenghai Yu. AutoPT: How far are we from the End2End automated web penetration testing?, 2024. arXiv:2411.01236.

- [21] Zheng Yu, Wenxuan Shi, Xinqian Sun, Zheyun Feng, Meng Xu, and Xinyu Xing. Patch validation in automated vulnerability repair, 2026. arXiv:2603.06858; PVBench: 40%+ of basic-test-validated patches fail rigorous testing.
- [22] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*, 2025. arXiv:2408.08926.
- [23] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents’ ability to exploit real-world web application vulnerabilities, 2025. arXiv:2503.17332.